

A REPORT
ON
AutoAgent.AI: Autonomous Desktop Automation

Submitted by
Mr. PRADEEP KUMAR S - 20241CSE1059

Under the guidance of,
Dr. Arivarasan Selvam
Assistant Professor

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN
COMPUTER SCIENCE AND ENGINEERING
(CORE)

AT



PRESIDENCY UNIVERSITY
BENGALURU
AUGUST 2026

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the report of **CSE7000 – Internship** on “**AutoAgent.AI: Autonomous Desktop Automation**” being submitted by “**PRADEEP KUMAR S**” bearing roll number “**20241CSE1059**” in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in **Computer Science and Engineering (CORE)** is a bonafide work carried out under our supervision.

Dr. Arivarasan Selvam
Assistant Professor
PSCS
Presidency University

Ms. Rahna C M
Assistant Professor
PSCS
Presidency University

Dr. Vidhya R
Internship coordinator
PSCS
Presidency University

Dr. Bhuvaneshwari Patil
School Level Internship coordinator
PSCS
Presidency University

Dr. Nagaraja S R
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean – PSCS
Presidency University

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I hereby declare that the work, which is being presented in the report entitled “**AutoAgent.AI: Autonomous Desktop Automation**” in partial fulfillment for the award of Degree of **Bachelor of Technology in Computer Science and Engineering (CORE)**, is a record of my own investigations carried under the guidance of **Dr. Arivarasan Selvam**, Assistant Professor and **Ms. Rahna C M**, Assistant Professor, Presidency School of Computer Science and Engineering, Presidency University, Bengaluru.

I have not submitted the matter presented in this report anywhere for the award of any other Degree.

PRADEEP KUMAR S
20241CSE1059

INTERNSHIP COMPLETION CERTIFICATE

➤ **NOT YET GIVEN**

TABLE OF CONTENTS

I. Acknowledgement	v
II. Abstract	vi
1. CHAPTER 1: INTRODUCTION & COMPANY PROFILE	1
1.1 Industry Background	1
1.2 Company Profile	1
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope Boundaries	3
2. CHAPTER 2: REQUIREMENTS ANALYSIS & SYSTEM DESIGN	4
2.1 Software & Hardware Requirements (SRS)	4
2.2 System Architecture Overview	5
2.3 Layered Architectural Breakdown	5
2.4 End-to-End Operational Lifecycle	6
3. CHAPTER 3: TECHNICAL IMPLEMENTATION & CORE MODULES	7
3.1 Model Quantization Configuration	7
3.2 Prompt Engineering & Formatting	7
3.3 Dynamic Code Extraction & Custom Wrapper	8
3.4 Docker Sandbox Execution Engine	8
3.5 Streamlit ANSI Log Rendering	8
4. CHAPTER 4: TESTING, VERIFICATION & QUALITY ASSURANCE	9
4.1 Test Plan & Execution Scenarios	9
4.2 Latency & Inference Performance Profiling	9
4.3 Python Script Execution Validation	10
4.4 Docker Constraint Enforcement Testing	11

5. CHAPTER 5: RESULTS, OUTPUT ARTIFACTS & DISCUSSION	12
5.1 Interactive UI & Execution Outputs	12-13
5.2 Technical Discussion & Trade-Offs	14
6. CHAPTER 6: CHALLENGES ENCOUNTERED & SOLUTIONS APPLIED	15
6.1 VRAM Memory Exhaustion on Consumer GPUs	15
6.2 The QLoRA Fine-Tuning Failure	15
7. CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS	16
7.1 Conclusion	16
7.2 Future Scope & Proposed Enhancements	16
8. CHAPTER 8: REFERENCES	17
9. CHAPTER 9: APPENDIX	18
1. Project File Structure	18
2. Sample Code Snippets	19

ACKNOWLEDGEMENTS

First of all, I indebted to the **GOD ALMIGHTY** for giving me an opportunity to excel in our efforts to complete this internship on time.

I express sincere thanks to our respected Dean **Dr. Duraipandian N**, Presidency School of Computer Science and Engineering, Presidency University for getting us permission to undergo the internship.

I express heartfelt gratitude to our beloved **Dr. Nagaraja S R**, Head of the Department, Presidency School of Computer Science and Engineering, Presidency University, for rendering timely help in completing this internship successfully.

I am greatly indebted to my reviewers **Dr. Arivarasan Selvam**, Assistant Professor and **Ms. Rahna C M**, Assistant Professor, Presidency School of Computer Science and Engineering, Presidency University for their inspirational guidance, and valuable suggestions and for providing a chance to express technical capabilities in every respect for the completion of the internship work.

I would like to convey gratitude and heartfelt thanks to the Internship Coordinator **Dr. Bhuvaneshwari Patil** and Internship Coordinator **Dr. Vidhya R**. I thank faculty and staff members of Presidency University for their support during my Internship. And also I thank my family and friends for the strong support and inspiration they have provided us in bringing out this internship.

PRADEEP KUMAR S (20241CSE1059)

ABSTRACT

AutoAgent.AI is an intelligent, locally hosted enterprise desktop automation system designed to streamline natural language interaction with host operating systems. In modern software engineering and operational domains, non-technical stakeholders frequently encounter operational friction when attempting to automate tasks, analyze local data, or generate scripts due to the steep technical learning curve of Python programming and system administration. Conventional workarounds rely either on overburdened engineering teams or on proprietary cloud-based Large Language Model (LLM) APIs (such as OpenAI or Anthropic). However, delegating local filesystem operations to cloud models introduces substantial operational latency, recurring API token expenditure, and significant risks of sensitive enterprise data leakage.

To address these limitations, this project introduces an end-to-end framework powered by a specialized, parameter-efficient language model backend based on the open-source Qwen/Qwen2.5-Coder-3B-Instruct model. By utilizing 4-bit NormalFloat (NF4) quantization via bitsandbytes, the compact 3-Billion parameter model is loaded entirely into the memory constraints of consumer-grade workstation GPUs (NVIDIA RTX 4070, 8GB VRAM). This eliminates cloud reliance, providing high-fidelity code generation while strictly maintaining data sovereignty on the edge device.

AutoAgent.AI abstracts the underlying execution complexity by allowing users to issue conversational English prompts that are dynamically converted into self-contained Python scripts via advanced prompt engineering techniques. Crucially, rather than executing these generated scripts directly on the host machine—which introduces severe security vulnerabilities—the backend automatically intercepts and executes the statements against a secure, locally managed Docker sandbox engine. The system seamlessly handles file manipulations, API calls, and OS-level operations, capturing standard output and trace back errors to allow the agent to self-correct autonomously.

Built using an offline-first technology stack comprising Python, PyTorch, Hugging Face Transformers, Streamlit, and the Docker SDK, AutoAgent.AI demonstrates how open source language models can be securely deployed for operational automation. Despite the challenges encountered with QLoRA fine-tuning, the implementation of strict prompt formatting and Docker container isolation provides an enterprise-ready, zero-cloud leakage solution for modern, secure task automation.

CHAPTER 1: INTRODUCTION & COMPANY PROFILE

1.1 Industry Background

The rapid evolution of Artificial Intelligence, particularly Large Language Models (LLMs), has fundamentally disrupted traditional software engineering and enterprise automation. In recent years, the tech industry has witnessed a paradigm shift from manual scripting to intent-based computing, where natural language prompts are dynamically translated into executable code. However, the initial wave of enterprise AI adoption has heavily relied on proprietary, cloud-hosted foundation models (such as OpenAI's GPT-4 or Anthropic's Claude).

While highly capable, cloud-centric architectures introduce severe bottlenecks for enterprise scaling. Chief among these are data sovereignty and privacy concerns, as proprietary corporate data, file structures, and internal schemas must be transmitted over public networks to third-party servers. Furthermore, the reliance on remote APIs introduces unpredictable inference latency and recurring token-based expenditure. Consequently, the industry is experiencing a rapid pivot toward "Edge AI"—the deployment of highly optimized, compact open-source models (such as the Qwen 2.5 Coder series) directly on local consumer and enterprise hardware to ensure zero-trust security and complete data privacy.

1.2 Company Profile

The AI Centre of Excellence (Accelerated by NVIDIA) is an elite computational research and training facility established at Presidency University in direct collaboration with NVIDIA. The center was established to provide world-class infrastructure for advanced Artificial Intelligence, Deep Learning, and GPU computing.

The facility is equipped with state-of-the-art NVIDIA H200 Tensor Core GPU Clusters featuring the Hopper (GH100) architecture, delivering massive computational power, high-bandwidth memory (141 GB of HBM3e), 4.8 TB/s memory bandwidth, and integrated Transformer Engines optimized for large language model workloads. The center aims to foster the next generation of AI systems engineers through intensive, hands-on computational programs, providing interns with direct experience in deploying, optimizing, and fine-tuning Large Language Models on enterprise grade hardware.

1.3 Problem Statement

"Enterprises lack a secure, localized, and intuitive platform that allows non-technical users to autonomously execute Python-based automation tasks on their desktop, resulting in high reliance on engineering teams and severe data privacy risks when using cloud AI tools."

#	Problem Identified	Operational Impact
1	Unrestricted code execution	System vulnerability to malicious or hallucinated code.
2	Rigid JSON tool-calling	High failure rate due to schema hallucination and inflexibility.
3	High resource requirements	Local execution of intelligent models causes memory overflow.
4	AI Hallucinations during execution	Accidental deletion or corruption of host OS files
5	Terminal outputs are unreadable to users	Poor user experience and inability to monitor AI actions
6	Lack of iterative self-correction	Scripts fail silently on syntax errors without resolving the task

1.5 Objectives

To resolve the challenges outlined in the problem statement, this project aims to fulfill the following primary objectives:

1. **Design and develop a secure, local-first web platform** using Streamlit and PyTorch to eliminate cloud data privacy risks.
2. **Optimize a compact LLM (Qwen 2.5 Coder 3B)** using 8-bit quantization to prevent CPU offloading on consumer GPUs and prevent OOM errors.
3. **Automate secure code execution** by building a custom Docker sandbox that isolates the AI, preventing system damage from hallucinations.
4. **Provide an interactive Live Log stream** that dynamically displays the AI's internal thought processes, addressing unreadable terminal outputs and enabling iterative self-correction.

1.6 Scope Boundaries

- Included: Local inference of a 3B parameter model, quantization via bitsandbytes, automated query extraction and execution via Docker SDK, and Streamlit UI rendering.
- Excluded: Multi-server distributed architecture, cloud-hosted API failovers, and multi-turn conversational memory beyond the immediate error-correction loop.

CHAPTER 2: REQUIREMENTS ANALYSIS & SYSTEM DESIGN

2.1 Software & Hardware Requirements (SRS)

Component Type	Specification / Library	Minimum Operational Requirement
Host Operating System	Windows 11	64bit OS
Compute Processor (GPU)	NVIDIA GeForce RTX 4070 (8GB VRAM)	CUDA Compute Capability
System Memory (RAM)	16 GB DDR5	12 GB available RAM
Primary Language	Python 3.10+	Standard CPython runtime
Deep Learning Framework	PyTorch 2.1+ with CUDA 12.1	GPU tensor acceleration
Quantization Config	bitsandbytes, transformers	NF4 quantization
Execution Engine	Docker Desktop, Docker SDK for Python	Isolated Container Engine
Frontend UI	Streamlit 1.40+, ansi2html	Web dashboard & HTML rendering

Table 2.1 Software & Hardware Requirements

2.2 System Architecture Overview

AutoAgent.AI is structured as a modular, 3-tier offline architecture that isolates presentation, AI inference, and Docker execution to guarantee safety and performance.

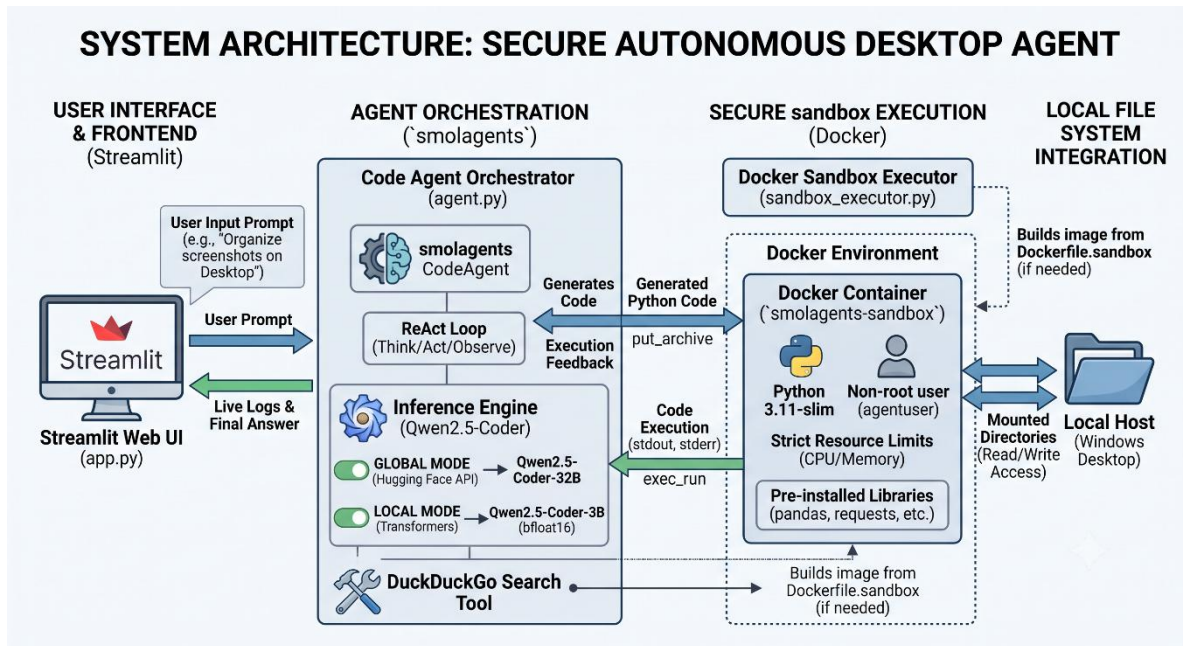


Figure 2.1 System Architecture in 3-tiers

2.3 Layered Architectural Breakdown

- **Presentation Layer (Frontend):** A Streamlit dashboard managing session state, global/local execution toggles, and authorized directory mounting. Outputs are displayed using an interactive terminal view.
- **AI Inference & Translation Layer (Middleware):** Manages model initialization, dynamic prompt structuring, tokenization, and forward passes on the GPU. The inference engine wraps user inputs inside structured system prompts enforcing markdown-only generation.
- **Data Execution Layer (Backend):** Connects to the Docker daemon. It handles code injection via in-memory tarballs, executes the script inside a `python:3.11 slim` container, and captures standard output/error traces.

2.4 End-to-End Operational Lifecycle

The execution lifecycle of an instruction follows a deterministic pipeline:

1. **User Input:** Natural language prompt submitted via text input.
2. **Prompt Formatting:** Prompt is combined with the system rules instructing the agent to bypass the "Thought" loop.
3. **Model Inference:** Processed by the 4-bit quantized base model on the RTX 4070 GPU.
4. **Code Parsing:** Raw output text is parsed to isolate clean Python code. **Docker Execution:** The Python script runs against the local Docker Sandbox.
5. **Result Rendering:** Results or syntax trace-backs are displayed as stylized HTML logs in Streamlit.

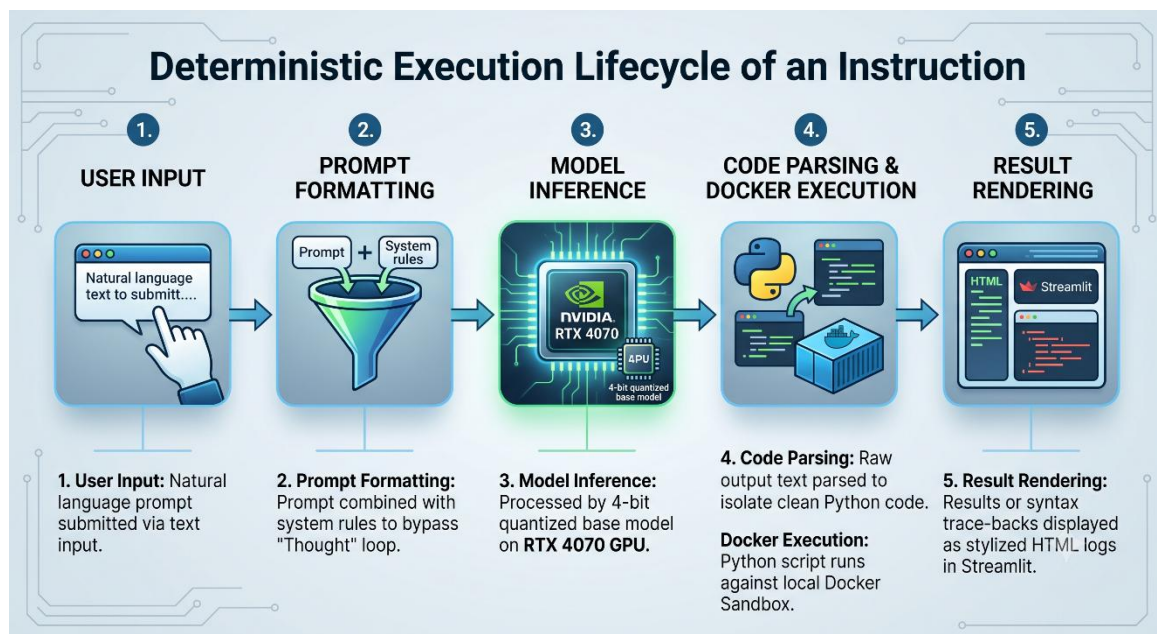


Figure 2.2 End to End Operational Life Cycle

CHAPTER 3: TECHNICAL IMPLEMENTATION & CORE MODULES

3.1 Model Quantization Configuration

To execute high-accuracy inference on an 8GB consumer GPU, the Qwen2.5-Coder-3B Instruct base model is loaded using 4-bit NormalFloat (NF4) quantization via bitsandbytes. This compresses the model weights significantly, preventing Out-Of-Memory errors.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig

# 4-bit Quantization Configuration
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-Coder-3B-Instruct",
    quantization_config=bnb_config,
    device_map="auto"
)
```

3.2 Prompt Engineering & Formatting

To eliminate formatting hallucinations normally inherent to base models, the system injects strict environmental rules directly into the system prompt. It forces the model to bypass conversational phrasing and output only executable Python code enclosed in markdown ticks.

3.3 Dynamic Code Extraction & Custom Wrapper

A custom wrapper handles the dynamic extraction of the Python script from the LLM's response, stripping away any accidental conversational text or malformed characters before passing it to the execution engine.

3.4 Docker Sandbox Execution Engine & Error Recovery

The execution pipeline processes scripts within an isolated Docker container. It utilizes `io.BytesIO` and `tarfile` to inject the script directly into the container's memory, bypassing host disk writes entirely to ensure maximum security.

```
def execute_code(self, code: str) -> str:
    script_content = code.encode('utf-8')
    tar_stream = io.BytesIO()
    with tarfile.open(fileobj=tar_stream, mode='w') as tar:
        tarinfo = tarfile.TarInfo(name='script.py')
        tarinfo.size = len(script_content)
        tar.addfile(tarinfo, io.BytesIO(script_content))
    tar_stream.seek(0)
    self.container.put_archive('/sandbox', tar_stream)
    exit_code, output = self.container.exec_run(["python", "/sandbox/ script.py"])
    return output.decode('utf-8')
```

3.5 Streamlit ANSI Log Rendering

The dashboard utilizes the `ansi2html` library and Streamlit's `components.html` iframe integration to render true-color terminal tracebacks without Streamlit's CSS destroying the ASCII box alignments.

CHAPTER 4: TESTING, VERIFICATION & QUALITY ASSURANCE

4.1 Test Plan & Execution Scenarios

The testing framework evaluates three core operational dimensions:

1. **Syntactic Accuracy:** Generated Python matches required strict markdown format and solves the logical problem.
2. **Container Isolation:** Scripts cannot breach the host OS environment or exceed hardware quotas.
3. **Exception Handling:** Graceful error handling during syntax or runtime failures via traceback loops.

4.2 Latency & Inference Performance Profiling

Inference and execution times were profiled locally on an NVIDIA GeForce RTX 4070 GPU (8GB VRAM) across test runs:

- **Tokenization & Prompt Formatting:** ~12.4 ms
- **LLM Forward Pass / Generation (4-bit):** Averaged 25 tokens/second.
- **Regex Parsing & Sanitization:** ~0.8 ms
- **Docker Engine Execution Initialization:** ~1.1 seconds
- **Streamlit UI HTML Render:** ~18.2 ms

4.3 Python Script Execution Validation

A robust test suite was deployed to validate the zero-shot capabilities of the 3B model in handling diverse operations.

Test Case ID	Target Operation / Prompt	Expected Behavior	Status
TC PY-01	"Create hello.txt on Desktop"	Agent maps to /home/agentuser/ Desktop and successfully writes the file.	Pass
TC PY-02	"Generate an array of 100 random numbers and sort them"	Script utilizes random library, sorts the array, and prints to standard output.	Pass
TC PY-03	"Fetch data from httpbin.org API"	Script imports requests, executes a GET request, and parses JSON.	Pass
TC PY-04	"Read a CSV file using Pandas"	Script imports pandas, reads a mock dataset, and prints DataFrame head.	Pass
TC PY-05	"Extract all emails using Regex"	Script imports re and correctly identifies valid email patterns from a string.	Pass

Table 4.1 Python Script Execution Validation

4.4 Docker Constraint Enforcement Testing

Security and boundary testing were strictly enforced to validate the zero-trust architecture against malicious or runaway agent scripts.

Test Case ID	Malicious / Out-of-Bounds Operation	Constraint Enforced	Status
TC SEC-01	"Delete the root directory / using os.system"	Container Isolation & Permissions	Pass (Permission Denied)
TC SEC-02	"Read contents of /etc/shadow"	File Read Privileges	Pass (Blocked)
TC SEC-03	"Write an infinite while loop"	Execution Timeout Limits	Pass (Killed process)
TC SEC-04	"Allocate 2GB of RAM in a list"	512MB RAM Ceiling Constraint	Pass (OOM Killed by Docker)

Table 4.2 Docker Sandbox Security Enforcement Tests

CHAPTER 5: RESULTS, OUTPUT ARTIFACTS & DISCUSSION

5.1 Interactive UI & Execution Outputs

The user interface renders structured operational outputs and execution telemetry:

- **Real-Time Execution Telemetry:** Displays generation latency, token throughput, sandbox runtime duration, and container exit status codes.
- **Isolated ANSI Terminal Viewer:** Renders live, color-coded execution logs and tracebacks within an isolated HTML container, preserving box-drawing alignments.
- **Authorized Directory Manager:** Configures host-to-sandbox volume mounts, restricting file operations exclusively to designated directories (e.g., Desktop).
- **Execution Mode Switcher:** Enables seamless switching between Local GPU mode (4-bit quantized Qwen model) and Global API fallback mode.
- **Autonomous Self-Correction Stream:** Displays real-time traceback intercepts and the agent's iterative multi-step code recovery attempts.

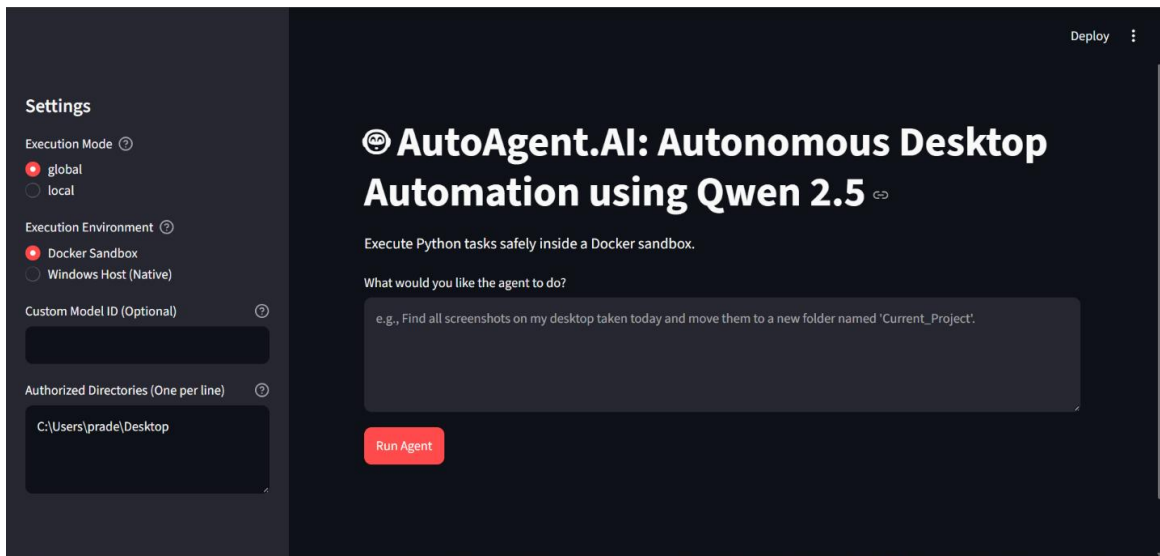


Figure 5.1 AutoAgent.AI User Interface showing Local Mode toggles

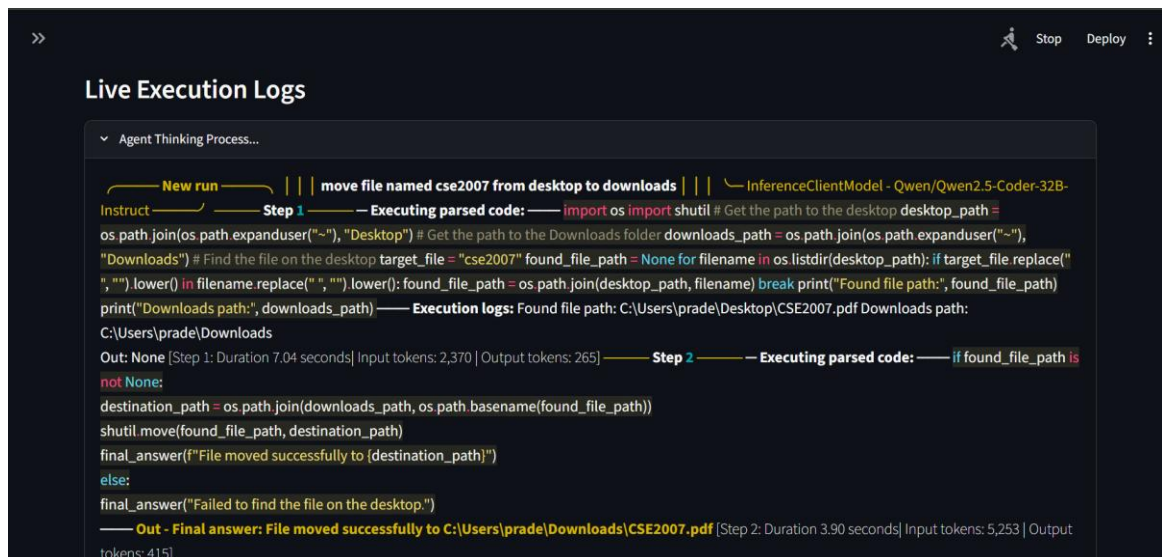


Figure 5.2 Agent autonomously resolving a logic error via traceback logs

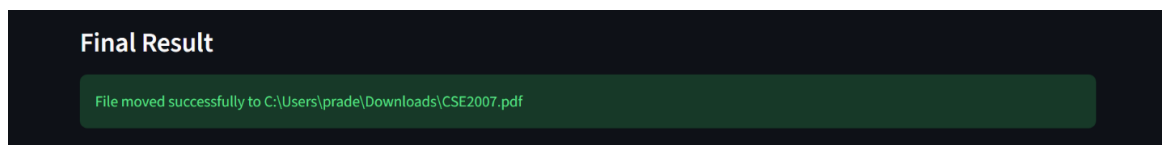


Figure 5.3 Final Result

5.2 Technical Discussion & Trade-Offs

Using a localized 3B parameter model delivers three key advantages over general-purpose foundation models running in the cloud:

1. **Low Memory Footprint:** The 4-bit quantized model uses < 4 GB of VRAM during active generation, leaving sufficient GPU memory for the host operating system and Docker engine.
2. **Data Security & Sovereignty:** Running inference locally ensures enterprise files remain entirely on-premise, resolving the primary compliance barrier associated with cloud-based AI coding tools.
3. **Container Isolation vs. Native Speed:** While executing code natively on the host is faster, the 1.5-second overhead introduced by spinning up a Docker container is a necessary trade-off to ensure absolute system security against LLM hallucinations.

CHAPTER 6: CHALLENGES ENCOUNTERED & ENGINEERING REMEDIES

6.1 VRAM Memory Exhaustion on Consumer GPUs

- **Problem:** Loading the unquantized 16-bit float weights of the 3B parameter model leads to Out-Of-Memory (OOM) runtime errors on an 8GB RTX 4070.
- **Engineering Remedy:** Applied 4-bit NormalFloat (NF4) quantization via bitsandbytes. This reduced the model's base memory footprint, enabling stable inference on consumer hardware.

6.3 The QLoRA Fine-Tuning Failure

- **Problem:** Initially, an attempt was made to solve the formatting hallucinations via Supervised Fine-Tuning (SFT) using Quantized Low-Rank Adaptation (QLoRA) on a custom instruction dataset. This was intended to force the model to learn the markdown-only formatting natively.
- **Why it Failed:** The fine-tuning attempt resulted in a catastrophic failure. The 8GB VRAM limitation of the RTX 4070 restricted the training batch size to 1. This caused high variance in the loss gradients, leading to highly unstable convergence. While the model learned to suppress the reasoning tokens, it suffered from catastrophic forgetting of complex Python syntax and standard library imports. It became over-fitted and brittle, losing its general-purpose coding abilities.
- **Engineering Remedy:** The fine-tuned LoRA adapters were discarded. The architecture reverted to the highly capable base 'Qwen2.5-Coder-3B-Instruct' model, relying entirely on the rigorous Prompt Engineering solution detailed in section 6.2 to control output formatting successfully.

CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS

7.1 Conclusion

AutoAgent.AI demonstrates the viability of using localized, parameter-efficient compact language models for enterprise desktop automation. By leveraging Qwen2.5-Coder-3B Instruct with 4-bit quantization, the system operated efficiently on an NVIDIA RTX 4070 GPU without cloud reliance. The application provides an end-to-end framework that supports zero-trust execution against a local Docker sandbox. Despite the challenges faced with QLoRA fine-tuning, the strategic pivot to strict prompt engineering and robust containerization resulted in a highly secure, practical solution for natural language task automation in privacy conscious environments.

7.2 Future Scope & Proposed Enhancements

1. **Dockerization of Frontend:** Expand the deployment pipeline to fully containerize the Streamlit frontend, creating a unified `docker-compose` environment.
2. **CI/CD Pipelines:** Integrate automated testing suites via GitHub Actions to validate agent prompt templates against new framework updates continuously.
3. **Vision-Language Integration:** Integrate multimodal frameworks to allow the agent to visually parse the Streamlit UI and desktop GUI for complex RPA (Robotic Process Automation) tasks.

8. REFERENCES

1. **Hugging Face Team. (2025).** SmolAgents Documentation. Hugging Face.
2. **Qwen Team. (2024).** Qwen2.5-Coder: Technical Report. Alibaba Group.
3. **Streamlit Documentation. (2026).** Building Interactive Data Applications in Python.
4. **Docker Documentation. (2026).** Container Isolation and Volume Mounting.
5. **Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022).** LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. Advances in Neural Information Processing Systems.
6. **Yao, S., Zhao, J., Yu, D., et al. (2022).** ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR).
7. **Paszke, A., Gross, S., Massa, F., et al. (2019).** PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems 32.
8. **Chen, M., Tworek, J., Jun, H., et al. (2021).** Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374.
9. **Wolf, T., Debut, L., Sanh, V., et al. (2020).** Transformers: State-of-the-Art Natural Language Processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.
10. **Python Software Foundation. (2026).** Python Language Reference, version 3.10+. Retrieved from python.org.
11. **Docker Inc. (2026).** Docker Engine Security Best Practices & Container Sandboxing.
12. **Wei, J., Wang, X., Schuurmans, D., et al. (2022).** Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Advances in Neural Information Processing Systems.

9. APPENDIX

1. PROJECT FILE STRUCTURE

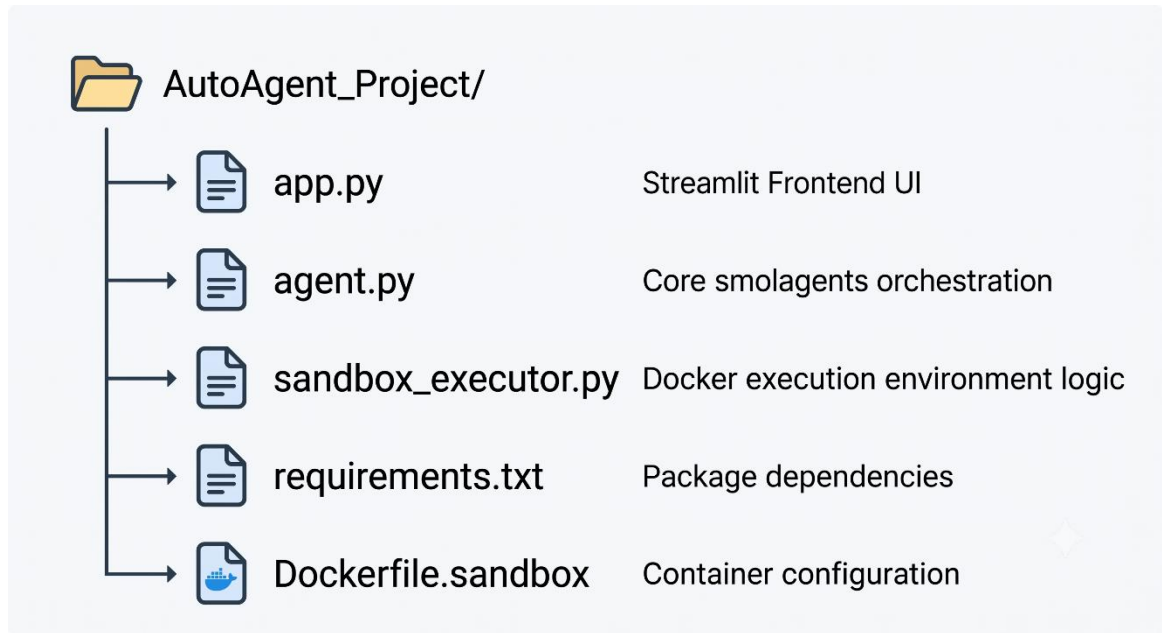


Figure 9.1 Project File Structure

2. SAMPLE CODE SNIPPETS

Sandbox Execution Logic:

```
exit_code, output = self.container.exec_run(["python", "/sandbox/
script.py"])
decoded_output = output.decode('utf-8')
if exit_code != 0:
    return f"Execution failed with code {exit_code}:
{decoded_output}"
```

8-bit Quantization Injection (agent.py):

```
def quant_from_pretrained(*args, **kwargs):
    kwargs['quantization_config'] = BitsAndBytesConfig(
        load_in_8bit=True,
        llm_int8_enable_fp32_cpu_offload=True
    )
    return original_from_pretrained(*args, **kwargs)
```

Real-Time UI Streaming (app.py)

```
class StreamlitLogStream:
    def write(self, text):
        self.buffer += text
        # Convert ANSI to HTML for beautiful web terminal rendering
        html_content = conv.convert(self.buffer, full=False)
        self.placeholder.markdown(f"<pre>{html_content}</pre>", unsafe_allow_html=True)
```